
COMP2511

— Tutorial 2 —

Last Week's Tutorial

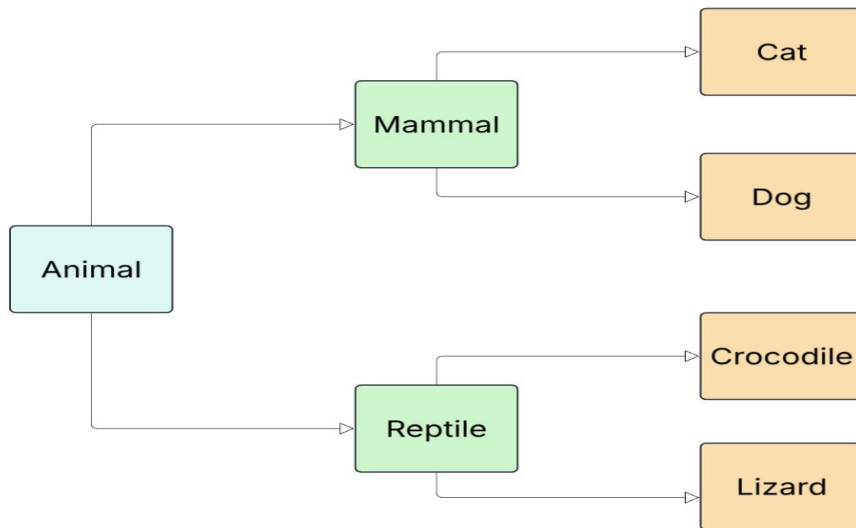
- Course Introduction
- Introduction to Java
- Introduction to Classes

This Week's Tutorial

- Inheritance
- Method Overriding
- Interfaces and Abstract Classes (if time permits)
- Polymorphism (if time permits)

Inheritance

- Intuitively, inheritance is about reusing existing classes to create new classes. Think of these new classes as **extensions** of another class.



Inheritance

- **Inheritance** means using an existing class as a basis for creating a new class, by making the new class have a copy of **every** field and method from the original class.
 - The new class is referred to as the **subclass/child class**, while the class being inherited from is referred to as the **superclass/parent class**.
 - Inheritance allows us to reuse code and extend upon existing systems.
 - If you were modelling two related classes A and B with a lot of shared attributes, you could capture all of the shared attributes in a parent class and make A and B inherit this parent class (**less repetition**).
 - If you had to add behaviour on top of an existing class, you could create a new subclass of the existing class, which contains the new functionality (**extensibility**).

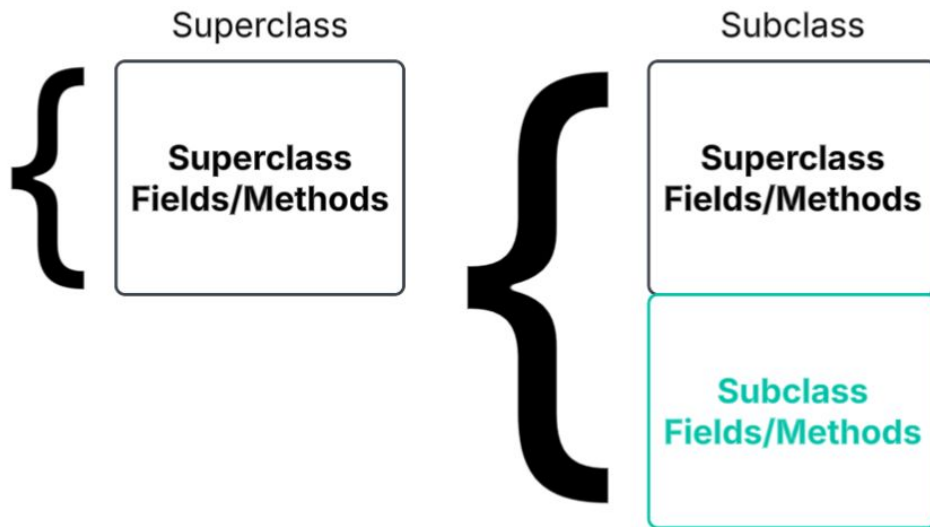
Inheritance

- Inheritance enforces an **'is-a'** relationship between a subclass and its superclass.
 - **If B is a subclass of A, then an instance of B is also an instance of A.** Use this as a test to determine if a inheritance relationship makes sense, e.g. a Cat is an Animal.
 - A subclass should be able to do the same things as its superclass, and more.
- All classes in Java are subclasses of the `Object` class.
- In Java, the **extends** keyword makes a class inherit from another.

```
public class Animal {  
    // some animal fields and methods ...  
}  
  
class Cat extends Animal {  
    // some cat fields and methods ...  
}
```

Inheritance

- Conceptually, the subclass is the parent class with more fields/functions added on top.



Quiz: Inheritance

- Suppose Cat and Dog are subclasses of Animal. Are each of the following valid?

```
public static void main(String[] args) {  
    Animal a = new Cat();  
}
```

Yes; all Cats are Animals.

```
public static void main(String[] args) {  
    Cat c = new Animal();  
}
```

No; not all Animals are Cats.

```
public static void main(String[] args) {  
    Animal[] animals = { new Cat(), new Dog() };  
}
```

Yes; an array/list of Animals can store any type of Animal.

Live Example: JavaDoc and Inheritance

- Review the **Employee** class in src/employee, which has been documented with JavaDoc.
 - How do you write JavaDoc?
 - Should code should always have comments/JavaDoc?
 - What is meant by the term "self-documenting code"?
- Create a **Manager** class that is a subclass of Employee and has a field for the manager's hire date.
 - What constructor(s) should we define for the Manager class?
 - Demonstrate how VSCode can generate getters and setters automatically.
 - Why might adding certain getters and setters be bad design?

Type-Checking in Java

- **Recall:** If B is a subclass of A, instances of B are instances of A.
 - This also applies for inheritance that goes deeper down (*transitivity*). For example, if C was a subclass of B, then instances of C are instances of A (and B, and C).
- If we want to check if an object is an instance of A **or any subclasses of A**, we use the **instanceof** keyword.
 - For example: `a instanceof A` returns true if the object a is of type A or any of its subclasses, and false otherwise.
- If we want to make an exact comparison on an object's class ignoring subclass relationships, we can compare using the `getClass()` method.
 - For example: `a.getClass() == b.getClass()` returns true if a and b are instances of the same exact class, and false otherwise.
 - This is rarely used, because functions that work with a parent class should also work with any of its subclasses.

Method Overriding

- **Recall:** A subclass inherits **all** of its superclass' fields and methods. Private fields/methods cannot be accessed, but are technically still there.
 - If A defines a method **doSomething()** and B extends A, then **doSomething()** can also be invoked from instances of B.
- A subclass can provide its own implementation of a method inherited from its superclass, which **overrides** the original functionality.
 - You need to use the same exact **method signature** in the subclass, i.e. the same method name and parameter types in the same order.
- All overridden methods should have the **@Override** tag on top.
 - This is not strictly enforced by the compiler, but is best practice. It prevents bugs by notifying you if you are trying to override a method that does not exist in the parent class or wrote the wrong method signature.

Quiz: Method Overriding

- What do you think the following code will output?

```
class A {  
    public void print1() {  
        System.out.println("Hello from A!");  
    }  
  
    public void print2() {  
        System.out.println("Hello again from A!");  
    }  
}  
  
class B extends A {  
    @Override  
    public void print1() {  
        System.out.println("Hello from B!");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        A a = new A();  
        B b = new B();  
        a.print1();  
        b.print1();  
        b.print2();  
    }  
}
```

Quiz: Method Overriding

- **a.print1()** prints "Hello from A!" as expected.
- **b.print1()** prints "Hello from B!" since this method has been overridden.
- **b.print2()** prints "Hello again from A!" since this method has not been overridden.

```
Hello from A!  
Hello from B!  
Hello again from A!
```

Live Example: Method Overriding (Pt. 1)

- **Recall:** All classes in Java are subclasses of the Object class.
- One inherited method from Object is **toString()**.
 - What does Object's implementation of **toString()** do?
 - What information would be useful to include in the output of **toString()**?
- Override the **toString()** method in the Employee and Manager classes defined earlier. Consider how code from Employee's **toString()** can be reused while writing Manager's **toString()**.

Live Example: Method Overriding (Pt. 2)

- What is a suitable criterion for two objects to be considered equal?
 - If two objects are instances of the same exact class and have all corresponding fields equal, we can consider them to be equal.
 - There are other ways to define equality, but we will take the above as the definition.
- Does the `==` operator abide by this definition of equality between objects?
If not, how does it actually determine equality?
- Another method all classes inherit from `Object` is **`equals()`**.
 - What does `Object`'s implementation of **`equals()`** do?
- Override the **`equals()`** method in the `Employee` and `Manager` classes.
Consider how code from `Employee`'s **`equals()`** can be reused while writing `Manager`'s **`equals()`**.

Abstract Classes

- An **abstract class** is a class that *cannot be instantiated*.
- They allow methods without concrete implementations to be declared, called **abstract methods**.
- Abstract classes act as templates for a common structure between any subclasses that derive off it.
 - This is useful for when you want to create a parent class capturing a bunch of common fields and methods, but this class doesn't make sense as a concrete object (e.g. Animal).
 - Concrete classes that inherit abstract methods must provide the concrete implementations for those methods.
 - Concrete implementations can still be defined within abstract classes.

Interfaces

- An **interface** is a *contract* prescribing a list of methods. Any class that **implements** the interface must provide a concrete implementation for these prescribed methods.
- Interfaces cannot be instantiated.
- Each class in Java can have at most one superclass, but can implement as many interfaces as required.

Polymorphism

- **Polymorphism** is the ability to use a **common interface** across **different types/classes** to invoke certain functionality, regardless of how that functionality is implemented in each of the classes.
 - In other words, it is the ability to call the same function on different objects.
 - Inheritance and interfaces are the ways we can prescribe the methods that can be called by a group of types, allowing us to use polymorphism.
 - Polymorphism captures the essence of well designed object-oriented code; it provides a simple yet flexible way to interact with objects.

Live Example: Polymorphism

- Look at the code in the **src/pets** package, which models different types of pets (birds, cats and dogs)
 - When does it make sense for a class to be abstract?
 - What is the difference between an abstract class and an interface? Why would you use one or the other?
 - Refactor the code to improve its quality.

Access Modifiers

Decreasing order of visibility



	public	protected	default	private
Same class	Yes	Yes	Yes	Yes
Same package	Yes	Yes	Yes	No
Subclass	Yes	Yes	No	No
Different package AND not a subclass	Yes	No	No	No

Quiz Questions

- How do you make a class be a subclass of another class in Java?
- What is the difference between type-checking using **instanceof** and **getClass()**?
- When overriding a method in a subclass, how do you access the superclass implementation of the method?
- What is the method signature of the method **public int addEdge(String u, String w, int weight)**?